Presenter guide — Can we defend this release?

Current 26-slide deck: 24 visible slides and two hidden backups. These are the deck's speaker notes and operating cues; detailed demo commands live in docs/DEMO-RUNBOOK.md. Historical cloud evidence and local tests are distinct from pending v2 cloud activation.

Demo entry/return pairs: 8/9, 13/14, 16/17, 20/21. Content ends at 80:00; Q&A runs to 90:00. No deployment, permission change or new paid run is implied.

Slide 01 — Can we defend this release?

Clock 00:00–01:00.

“A useful agent is the beginning, not the release decision. Today we follow one synthetic Volt & Co customer through build, prove, release, operate and improve.

“We will distinguish historical cloud evidence from the revised local code. New cloud activation, calibration, red-team results and promotion are still pending. That distinction is how we defend a release rather than merely demonstrate one.”

Slide 02 — A working demo is not a defensible service

Clock 01:00–04:00.

The hook.
Read the quote. Pause. "Building the agent is the easy part now. Operating it with trust is the hard part."
Walk the four questions quickly — they are the spine of the session:
1. Is it right? — We'll see an answer that sounded perfect and was wrong.
2. Is it safe? — We'll show controls firing, not a list of settings.
3. Can we prove what shipped? — The gate and the case file.
4. What did production teach us? — A real finding that escaped our tests, and how it came back.
"If you remember one thing: features are table stakes. Every vendor has agents, evals, tracing and guardrails. What customers can't do on their own is connect them into one engineering system. That's what we'll show."

Slide 03 — 90 minutes, one agent, two loops

Clock 04:00–05:00.

“The agenda follows the lifecycle rather than the feature catalog. Four demonstrations answer four questions: how was the answer produced, what does passing establish, can we defend the artifact, and what did operation teach us?

“The two bridges connect developer work with operation. Keep Demo 1 at 17:00, Demo 2 at 32:00, Demo 3 at 42:00 and Demo 4 at 60:00. Protect the ten-minute Q&A.”

Slide 04 — Two loops, joined by two bridges

Clock 05:00–10:00.

The mental model for the whole session. Spend the time here — everything hangs off this slide.
Inner loop (left): where one candidate is built and proven, in the DEV project. Fast, iterative, developer-owned. Build → Trace → Evaluate → Guard and optimize.
Outer loop (right): where a candidate becomes a service, in DEMO (our stand-in for production). Release → Operate → Watch → Learn.
Bridge 1: a candidate doesn't cross because someone feels good about it. It crosses with evidence, through a gate, with a named approver.
Bridge 2: production is a source of test cases, not just alerts. A finding comes back as a test before it comes back as a fix.
"Most teams have both loops. Very few have both bridges. The bridges are where trust is engineered."
Point out the small grey Foundry capability names: "That's where each Foundry capability lives in the story. We'll meet them in this order."

Slide 05 — Volt & Co needs a support agent it can defend

Clock 10:00–12:00.

Meet the customer. This one request is the golden request — it appears in every demo.
"Volt & Co sells chargers. One batch, CH-0722, has been recalled. A customer is past the normal 30-day return window and wants a replacement. The honest answer is: you're eligible for a human replacement assessment — not 'approved', not 'I've escalated it'."
The right-hand card is the most important thing on the slide: the agent's tools are read-only. It cannot create tickets, hand off, refund, ship or change orders. "Remember this list. In about 50 minutes, production will show us the agent promising something on this list."
Everything is synthetic — say so once, here, and don't repeat it.

Slide 06 — First decision: who owns the harness?

Clock 12:00–15:00.

Who owns the harness? This is the first SE takeaway.
Optional hook (workshop car analogy): the model is the engine; the harness is the car — tools, instructions, safety systems; the user is the driver.
"The one question that decides the runtime is: who owns the harness?"
- Prompt agent: you configure, Foundry runs the loop. Start here.
- Copilot harness: still a prompt agent — not a third agent type — with skills, managed shell and multi-step work, operated by Foundry. Preview.
- Hosted agent: your code and framework; Foundry hosts it, scales it and gives it an identity.
"Volt chose hosted for one reason you'll see in the next demo: a business rule — the decision must equal what the eligibility tool returned — has to be enforced in code, not requested in a prompt."
Do NOT quote preview dates, regions, pricing or parity. Do not show the private-preview AKS option.

Slide 07 — One governed path for every request

Clock 15:00–17:00.

“Follow the customer request through the hosted agent, curated Toolbox and governed model/tool routes to the business and policy backends. Toolbox curation is not a substitute for backend authorization.

“The saved golden DEMO request has 43 correlated spans. Those spans identify that request, not today's invocation and not every future request. DEV and DEMO have distinct projects and connections but share an account.

“The APIM contracts cover configured application routes, not every internal evaluation, IQ or Memory call. They are not a universal spending cap. Now we will inspect the request and its evidence.”

Slide 08 — Build and trace

Clock 17:00–25:00 · 1 minute transitions.

“Let us inspect the golden replacement request and distinguish the model's tool choices from the business decision.

[SWITCH OUT: use docs/DEMO-RUNBOOK.md Demo 1 only after its cloud prerequisites are met. Until then, open Recorded evidence.html → Released DEMO v1 → replacement-recall and identify it as historical. Do not expect serving v1 to contain the new renderer.]

“Compare identifiers, actual calls, outputs and policy references. A catalog entry is configuration; a matched call and result is execution evidence.

[Show the saved response's matching trace, then the current local evidence_contract.py. A fresh request needs its own correlated trace. Stop waiting after 30 seconds; never borrow another run's identity.]

[RETURN: come back to this card and advance once to slide 9.]

Slide 09 — The first evaluation: fluent, and wrong

Clock 25:00–27:00.

“The historical DEV baseline passed eight of nine cases. In the failing case the tool returned eligible while the response said recommendation. Fluent wording did not preserve the structured business meaning.

“The v1 grader checked the reviewed expected decision, required successful tool and reference. V2 additionally compares actual returned status and arguments. We retain independent expected outcomes so a broken tool and answer cannot pass simply by agreeing.”

Slide 10 — Code owns the decision; the model owns understanding

Clock 27:00–29:00.

“The historical repair changed instructions and achieved nine of nine. The newer application boundary is different: code derives the decision from this run's tool evidence and constrains the customer answer.

“The model still interprets the request, selects tools and extracts identifiers. The trade-off is templated wording and buffered customer text, not removal of the streaming protocol. The new code has local Agent and Responses-host proof; deployment remains pending.”

Slide 11 — Four checks, four different questions

Clock 29:00–31:00.

“Code checks exact expectations, arguments, outcomes and references. A model rubric checks the meaning of action claims. Human review supplies the expectation when language is ambiguous.

“Eight positive and negative calibration examples are authored, but they have not been calibrated in Azure. The red-team adapter is implemented; SDK installation and the actual content-risk scan remain pending. Neither is represented by a passing local unit test.

“The regression dataset retains the original nine expectations and adds two customer-language escalation cases. Eleven synthetic cases are a bounded regression suite.”

Slide 12 — Controls you can show, not just configure

Clock 31:00–32:00.

“The recorded guardrail marker produced a native content_filter block. Ordinary utility was exercised separately. That is specific evidence, not proof of every guardrail boundary.

“The completed Optimizer experiment retained its baseline at 0.75. No improved candidate was accepted or applied. That is different from the separate instructions-only business repair and promotion.

“A tool-boundary control remains a separately approved follow-up.”

Slide 13 — Prove it

Clock 32:00–40:00 · 1 minute transitions.

[SWITCH OUT: inspect the recorded DEV baseline and failed row, then the v2 code grader and action rubric. Run the existing local runtime/evaluation command from the runbook.]

“Local tests show the actual Agent tool loop and Responses stream enforce the output boundary. They do not prove a new Azure deployment.

[Show the recorded guardrail result and completed Optimizer result. Red-team scorecard: unavailable until a genuine scan succeeds; explain the planned check rather than inventing a result.]

[RETURN: come back to this card and advance once to slide 14.]

Slide 14 — One gate for every behaviour change

Clock 40:00–41:00.

“This is the designed release mechanism, not a claim that the cloud gate is activated. Credential-free checks run now. Live jobs require the activation variable, real environment reviewers, scoped identities and the required branch status.

“After approval, the gate deploys and evaluates an exact candidate. The release consumes its ZIP, dataset and grader identities, without rebuilding. Red teaming is a separate scan, not silently included in the business gate.”

Slide 15 — The case file: evidence you can reopen

Clock 41:00–42:00.

"The case file contains source and package identity, responses, native results and rubric judgments. Calibration is inside action-judgments.json, not a separate calibration.json file.

"Promotion and independent DEMO evidence are in the release artifact. A separately executed red-team scan has its own output. The workflows request 90-day retention; repository policy may limit it.

"Historical case files can be inspected now. Future v2 results must wait for actual executions."

Slide 16 — The gate

Clock 42:00–50:00 · 1 minute transitions.

[SWITCH OUT: open the workflow and docs/RELEASE-GATE.md. Run the local gate tests, then inspect the historical candidate and DEMO release receipts.]

“We can show rejected stale checks, mismatched lineage and missing approval controls locally. Those tests are not a completed live PR gate.

“The historical release proves the original ZIP and v1 outcomes. New failed/passing live runs and v2 promotion remain pending.

[After approved activation, replace this route with actual run URLs and artifacts from the runbook. Do not approve or deploy as an improvised stage action.]

[RETURN: come back to this card and advance once to slide 17.]

Slide 17 — In production, watch behaviour, not just uptime

Clock 50:00–54:00.

“Operation asks what happened and whether the release still meets its expectations. Traces help explain individual requests; evaluation measures defined criteria.

“Our prepared weekly workflow replays synthetic v1 cases against an explicitly pinned DEMO version. It is not enabled, it does not sample customer traffic, and it does not measure every action claim. Capture or scoring failures will require triage, not an automatic claim of a behavioral regression.

“Foundry Insights is a preview capability we can explain, but no Insights monitor was configured for this workshop.”

Slide 18 — The finding that escaped: a promise it could not keep

Clock 54:00–57:00.

“The recorded DEMO answer said it was escalating to human support. There was no handoff tool. The answer nevertheless scored 1.0 under v1's decision, required-tool and reference checks.

“The response ID matches the saved trace. That is our genuine outer-loop finding, unlike the earlier DEV mismatch. The annotation is a reviewed repository record, not a native Insights finding.”

Slide 19 — Production teaches development what to test next

Clock 57:00–60:00.

“A trace records what happened; a human supplies what should happen. Here the expectation is advice to contact support, not a claim that a handoff occurred.

“The two customer-language escalation cases and the action rubric make that expectation reviewable. Replaying the historical response proves that v1 missed the claim and the local v2 regression catches it.

“The code repair then needs the same separately activated release gate. AI-written changes do not get a different standard.”

Slide 20 — What did production teach us to test next?

Clock 60:00–70:00 · 1 minute transitions.

[SWITCH OUT: open the recorded DEMO response, matching trace and recordings/escalation-finding.json. Show the two new dataset cases.]

[Run the recorded-finding regression and the actual Agent/Responses-host tests from the runbook. Show the repair diff, not a fabricated live pass.]

"This is real recorded evidence and a local fail/fix/pass demonstration. The new cloud gate, calibration and promotion are still pending. The prepared weekly v1 check alone would not have detected this missing semantic criterion."

[RETURN: come back to this card and advance once to slide 21.]

Slide 21 — The decisions behind the architecture

Clock 70:00–74:00.

“Each row is a deliberate architecture choice. Hosted code lets us enforce the contract; curated tools narrow the capability set; IQ retrieval remains inspectable.

“The final renderer cites references supplied by business tools, not arbitrary model prose. Backend authorization is still required. Release and scheduled checks are implemented but awaiting cloud activation and proof.

“The feedback path is the part to take into a customer's design review: an observed failure becomes a reviewed test before the next release.”

Slide 22 — Three things to remember

Clock 74:00–76:00.

Say the three lines slowly. These are what people should repeat to a colleague tomorrow.
1. Who owns the harness? — the runtime decision.
2. A green badge is not evidence — the release discipline.
3. Production teaches development — the improvement loop.

Slide 23 — Take it to your customer: a POC that ends in evidence

Clock 76:00–80:00.

Make it actionable for SEs.
"Don't sell a demo. Sell a POC that ends with a case file the customer's own approvers signed."
Scope it to one journey and write down what the agent must never do — that list becomes your rubric. Build the gate in week one, not at the end. Prove it on their data with their reviewers. Decide with evidence.
Point to the repo and the Dev Pack. Then open Q&A.

Slide 24 — Questions

Clock 80:00–90:00.

Q&A: use the backup slides only when relevant.

Why a model? It understands the request and chooses tools; code owns the final business decision.
Are eleven cases enough? They are a regression suite, not a production-readiness certificate.
Does the live gate run now? Credential-free checks run; cloud evaluation and promotion require separate activation and approvals.
Is this hard production isolation? No: DEV and DEMO share an account.
Where is the red-team scorecard? No scan has completed; SDK installation is currently blocked.
Memory? Historical one-caller DEV proof exists. Cross-user isolation and the revised Memory-enabled response UX are not cloud-verified.

Slide 25 — What this session does and does not claim

Hidden Q&A backup.

Backup only. Explain synthetic data, bounded coverage, shared-account isolation limits, preview status and pending cloud evidence. No SLA, regional-processing or universal-safety promise.

Slide 26 — Memory: verified, and deliberately off in DEMO

Hidden Q&A backup.

Backup only. Historical Memory proof covered one authenticated DEV caller; Memory remains off in DEMO. The revised contract is always installed, so Memory-only requests return unavailable without business evidence. Do not imply the old cloud receipt verifies the revised UX.